Lab — Week 4: REST, Exceptions, Logging, Config, Actuator and Docker (Student)

Week	4 — Mon 12 October 2026
Main lecture	REST APIs, Exception Handling, Logging, Configuration, Actuator and Docker
Duration	120 minutes
Your project	`student-app/` (TODO stubs — you fill these in)
Companion files	`presenter.md` (instructor), `solution.md` (reference — released after the lab)

Context

You now turn CampusShop into a **containerised REST service**. Storage stays in-memory (file persistence is Week 5); the focus is the HTTP contract, **exception→status mapping**, structured logging, externalised configuration, the Actuator health endpoint, and shipping a **Docker image**. The domain, DTOs, mapper, repository, controller and config are **given**; you implement the service logic, the global exception handler and the Dockerfile.

How the pieces fit together

A request travels controller → service → repository; anything that goes wrong is translated to a stable JSON error in one central place:

```
POST /partner-orders → OrderController (given, validates with @Valid)
                     → OrderService (YOU: id, map, save, log, uniqueness)
                     → InMemoryOrderRepository (given)
       any exception → GlobalExceptionHandler (YOU: exception → status + ErrorResponse)
```

Because `@Valid` runs **before** the controller method body, invalid input never reaches your service — it surfaces as `MethodArgumentNotValidException`, which is why validation is handled in the exception handler, not in `OrderService`. Configuration follows the same "given, you use it" pattern: `application.yml` binds `app.orders.enforce-external-id-uniqueness` (and `SERVER_PORT`) with env-var defaults, so Task 6 changes behaviour without touching any code.

Where everything is

All under `student-app/src/main/java/de/campusshop/rest`:

Path	What it is	You
`web/OrderController`	The three endpoints (`POST /partner-orders`, `GET /orders/{id}`, `GET /orders`)	read
`web/GlobalExceptionHandler`	Central error mapping — the TODO lists all four handlers	Task 4
`service/OrderService`	`importOrder` / `findById` stubs	Tasks 2–3
`repository/InMemoryOrderRepository`	Map-backed storage (interface: `OrderRepository`)	read
`mapper/OrderMapper`	request → domain → response	read
`dto/*`, `model/*`, `exception/*`	Records, domain, `DuplicateOrderException` / `OrderNotFoundException`	read
`config/OrderProperties`	Binds the uniqueness flag from `application.yml`	read
`src/main/resources/application.yml`	Port, uniqueness flag, Actuator exposure	read
`Dockerfile` (project root)	Multi-stage skeleton with TODOs for both stages	Task 5
`src/test/java/…/OrderControllerTest`	The 4 tests — your definition of done	run

Learning objectives

- map domain failures to the correct HTTP status with `@ControllerAdvice` / `@ExceptionHandler`;
- return a **stable** `ErrorResponse` JSON for every failure;
- generate ids with `UUID.randomUUID()` and log with context (no secrets);
- resolve configuration from `application.yml`, environment variables and command-line args;
- expose `/actuator/health`; and **build, run, save and load** a Docker image.

Prerequisites

- Java 21 + Maven, Docker Desktop (**started** — check with `docker run hello-world`), and `curl`.
- Open `student-app/`; run `mvn test` — `healthIsUp` passes, the other three fail (red) until you implement the tasks. The red tests are the specification: read them first.

Tasks

1. **Boot check** — run the app; confirm `GET /actuator/health` returns `{"status":"UP"}` (given).

2. **`OrderService.importOrder`** — generate a UUID `orderId`, map → save → log at INFO; if `enforce-external-id-uniqueness` is on and the `externalOrderId` already exists, throw `DuplicateOrderException`.
3. **`OrderService.findById`** — return the mapped order, or throw `OrderNotFoundException`.
4. **`GlobalExceptionHandler`** — add `@ExceptionHandler` methods mapping: `MethodArgumentNotValidException` → `400 VALIDATION_FAILED` (with field errors), `HttpMessageNotReadableException` → `400 MALFORMED_REQUEST`, `OrderNotFoundException` → `404 ORDER_NOT_FOUND`, `DuplicateOrderException` → `409 DUPLICATE_ORDER`.
5. **Dockerfile** — complete the multi-stage build, then `docker build`, `run`, `save`, `load`.
6. **Config override** — run with `--app.orders.enforce-external-id-uniqueness=false` and with `SERVER_PORT=9090`; confirm the behaviour/port changes (config precedence).

`mvn test` is green when Tasks 2–4 are done. Task 5/6 are verified by the commands below.

How to run and check

```
cd student-app
mvn test
mvn spring-boot:run

curl http://localhost:8080/actuator/health
curl -i -X POST http://localhost:8080/partner-orders -H "Content-Type: application/json" -d '{
  "partnerOrderId":"22222222-2222-4222-8222-222222222222",
  "buyer":{"id":"33333333-3333-4333-8333-333333333333","fullName":"Alex","email":"a@b.com"},
  "items":[{"sku":"BOOK-1","name":"Clean Code","quantity":2,"unitPrice":32.50}]}'

# Docker delivery (after Task 5)
mvn clean package && docker build -t campusshop-rest:lab .
docker run -p 8080:8080 -e ENFORCE_EXTERNAL_ID_UNIQUENESS=false campusshop-rest:lab
docker logs <id>; docker save campusshop-rest:lab -o campusshop-rest-lab.tar
```

Stretch goals (if you finish early)

No new tests — verify each with `curl` and the Docker CLI:

1. **Catch-all handler** — add `@ExceptionHandler(Exception.class)` mapping any unexpected failure to `500 INTERNAL_ERROR` with the same stable `ErrorResponse` shape (and log it at ERROR with the stack trace — the only handler that should). Prove it by temporarily throwing a `RuntimeException` in a service method.
2. **`.dockerignore`** — add one (at minimum `target/`, `.idea/`, `*.iml`); rebuild and compare the build-context size Docker reports. Then inspect the image with `docker history campusshop-rest:lab` — which layer is biggest, and why does layer order matter for caching?
3. **`/actuator/info`** — expose real content: set `management.info.env.enabled=true` and add `info.app.name` / `info.app.version` properties; confirm `curl :8080/actuator/info` returns them, then override one via an environment variable in `docker run` (config precedence again).

Reflection questions

1. Which status code for invalid input, missing order, duplicate, and unexpected failure — and why not `500` for a duplicate?
2. Where does `@ControllerAdvice` run relative to the controller?
3. What should and should **not** appear in logs?
4. How does Spring resolve `application.yml` vs an env var vs `--arg`? Which wins?
5. Why is "works in IntelliJ" not a valid delivery? What does `docker save` / `load` prove?

Reading

Priority	Reading	Scope
Recommended	Spring REST, 2nd ed. (Varanasi, Bartkov), UoE library	ch. 3 RESTful Spring, ch. 5 Error handling
Recommended	Cloud Native Spring in Action (Vitale), UoE library	ch. 4, sections 4.1 and 4.2 (properties, profiles, externalised configuration); ch. 6, sections 6.1 and 6.2 (container images for Spring Boot)
Recommended	Spring Boot Reference: Externalized Configuration	up to and including "Type-safe Configuration Properties"
Recommended	Docker Docs: Docker concepts, the basics	"What is a container?" to "What is Docker Compose?"
Optional	Spring Framework Reference: Annotated Controllers	request mapping and handler methods
Optional	Spring Boot Reference: Actuator Endpoints	the health endpoint
Optional	MDN: HTTP response status codes	2xx and 4xx classes

Submission

Push the completed `student-app/` and the exported image tar as your tutor directs. All four tests pass.